Reporte de despliegue: API REST en contenedor - Equipo 11

TC5061 MLOps · Semana 8 · Repositorio: https://github.com/oscartrix/TC5061-Proyecto-Equipo11

Repo: https://github.com/oscartrix/TC5061-Proyecto-Equipo11

Ramas de trabajo: `feat/api-contenedor` (#34), `ci/api-contenedor` (#35), `feat/inferencia-onnx` (#36), `docs/entrega-semana-8` (#37) y `feat/entrega-semana-8` (#38)

Fecha de la auditoría: 2026-11-06

1. Punto de partida

Área	Estado en `main`	Brechas
Carga desde el Model Registry	`mi_bici/api/model.py` (PR #32): alias `champion` o versión fija con `MODEL_NAME`, `MODEL_ALIAS`, `MODEL_VERSION`	Solo funciona con un servidor MLflow accesible
Esquemas Pydantic	`schemas.py` (PR #31): `strict`, rangos, `zone` acotada, `is_weekend` coherente con `day_of_week`, 1 a 500 filas	Ninguna
Endpoints	PR #33 abierto: `/health`, `/model-info`, `/p1/predict`	Sin revisión; sin fusionar
Contenedor	`Dockerfile` levanta el servidor MLflow, copia `.git/` y no ejecuta uvicorn	No existe imagen de la API
CI	`calidad` + `pipeline` (semana 7 y registro de modelos)	No construye ni prueba la imagen
Documentación y evidencia	README con la API local	Sin instrucciones de contenedor ni capturas

2. Qué se hizo, por qué, implicaciones y siguientes pasos

2.1 Revisión del PR #33 (Chuy)

• **Qué se hizo:** se revisó el código de los endpoints (carga del modelo al arrancar, errores 422 y 500 sin detalle interno, recorte de predicciones negativas a 0), se corrieron en local las 107 pruebas de la API y se confirmó el CI en verde. Jimena aprobó y se fusionó a `main`.

• **Por qué:** la API es la base de la entrega; sin los endpoints no había nada que empaquetar, y la regla de `main` exige una revisión antes de fusionar.

• **Implicación:** `main` ya expone `/health`, `/model-info` y `/p1/predict` con validación estricta de entradas; el trabajo de esta semana se construye sobre ese código sin modificarlo.

• **Siguientes pasos:** ampliar las pruebas de la API con casos de lotes grandes y cabeceras inválidas, y agregar endpoints para P2 y P3 reutilizando `load_served_model`.

2.2 Modelo dentro de la imagen

• **Qué se hizo:** `scripts/export_model.py` descarga el champion de P1 del Model Registry a `models/p1_champion/` y guarda nombre, versión y corrida en `registry.txt`. `load_served_model` usa `MODEL_PATH` si existe y, si no, el registry como antes. Prueba nueva de la carga local.

• **Por qué:** la imagen debe funcionar igual en cualquier máquina; depender de un servidor MLflow accesible desde el contenedor hace el arranque frágil (red, puertos, alias cambiante).

• **Implicación:** la versión servida queda fija en la imagen y el contenedor arranca sin servidor MLflow. A cambio, cambiar de modelo exige exportar de nuevo y reconstruir la imagen. La carpeta del modelo no se versiona en git.

• **Siguientes pasos:** publicar la imagen en un registro de contenedores con la versión del modelo en la etiqueta, y automatizar la exportación cuando cambie el alias `champion`.

2.3 Containerfile de la API

• **Qué se hizo:** `Containerfile` con `python:3.13-slim`, `requirements-api.txt` (solo lo que usa la API: `mlflow-skinny` en lugar de MLflow completo, sin matplotlib, pillow, skops ni pyarrow), usuario sin privilegios y `HEALTHCHECK` a `/health`. Se conserva el `Dockerfile` existente, que sigue levantando el servidor de MLflow. Objetivos de Makefile `api-model`, `api-build` y `api-run`; `CONTAINER_ENGINE` cambia entre docker y podman.

• **Por qué:** el `Dockerfile` existente copia `.git/` y arranca el servidor de tracking, que no es lo que se necesita para servir predicciones; un archivo aparte evita mezclar los dos usos. Reducir dependencias baja el tamaño y la superficie de seguridad.

• **Implicación:** la imagen pesa 814 MB (1.16 GB antes de recortar dependencias), arranca sana en unos 10 s y se reconstruyó con `--no-cache` dando la misma predicción. Se probó con Docker; el mismo archivo sirve con Podman, cuyos comandos son equivalentes pero no se ejecutaron.

• **Siguientes pasos:** probar el `Containerfile` con Podman, reducir más la imagen con una construcción en dos etapas y escanearla en busca de vulnerabilidades.

2.4 CI del contenedor

• **Qué se hizo:** al final del job `pipeline` se exporta el champion recién registrado, se construye la imagen, se levanta el contenedor y se prueba `/p1/predict` con una fila válida (espera 200) y una inválida (espera 422). La respuesta se guarda como artefacto. Las pruebas, la seguridad y DeepChecks de la semana 7 ya corrían en `calidad` y `pipeline`.

• **Por qué:** una imagen que solo se prueba a mano puede romperse con cualquier cambio; el CI detecta antes de fusionar que la imagen no construye o que la API no responde.

• **Implicación:** cada PR a `main` valida de punta a punta, desde los datos hasta el contenedor respondiendo. El pipeline completo tarda unos 15 minutos. #35, #36 y #37 apuntaban a una rama y no a `main`, por lo que el workflow no se disparó solo; el CI de #35 se lanzó con `workflow_dispatch` y el de #34 y #38 corre por ser PR a `main`.

• **Siguientes pasos:** separar el job del contenedor para no repetir el entrenamiento, y publicar la imagen desde el workflow al fusionar a `main`.

2.5 Documentación y evidencia

• **Qué se hizo:** sección "API en contenedor" del README con comandos para Docker y Podman, variables, petición y respuesta de ejemplo y la estrategia de despliegue (inferencia en tiempo real). Registros y capturas de pruebas, construcción, contenedor y peticiones en `docs/evidencia/api/`, y una captura de Swagger con la API respondiendo desde el contenedor.

• **Por qué:** el servicio debe poder usarse y reproducirse por alguien que no participó en el desarrollo.

• **Implicación:** el README es la referencia única para construir, ejecutar y consumir la API. Las capturas de terminal se generaron a partir de los registros; la de Swagger es una captura real de la API en Docker.

• **Siguientes pasos:** repetir la captura desde el equipo de otro integrante y agregar la versión del modelo y la fecha de exportación al README.

2.6 Extra: ONNX Runtime

• **Qué se hizo:** `INFERENCE_BACKEND=onnx` sirve el champion con ONNX Runtime, con `scripts/export_onnx.py` (conversión y comparación contra sklearn), `scripts/benchmark_inference.py` y pruebas. Por defecto la API sigue con sklearn. Análisis completo en la sección 5.

• **Por qué:** explorar un servidor de inferencia más rápido sin cambiar el comportamiento actual de la API. OpenVINO no se integró: parte de ONNX y solo acelera en hardware Intel.

• **Implicación:** ONNX responde de 7 a 46 veces más rápido, pero con el HGB actual cambia la predicción en parte de las filas por usar float32. Por eso no es el motor por defecto.

• **Siguientes pasos:** definir la tolerancia aceptable de diferencia, probar con Ridge, y refactorizar `model.py` para construir una imagen que no incluya MLflow ni sklearn.

3. Resultados

• Pruebas: 335 pasan (`evidencia/01_pytest.txt`).

• Imagen `mibici-api`: 814 MB; contenedor `healthy` en unos 10 s sin servidor MLflow (`02_build.txt`, `03_contenedor.txt`).

• Petición válida: `200` con `{"model_name":"mibici-equipo11-p1-demanda","model_version":"1","predictions":[10.182341634926482]}`; petición con `month: 13` y campos faltantes: `422` (`04_peticion.txt`).

• Champion: HGB v1, entrenado en esta máquina con los datos de `data/processed/`; los valores pueden diferir en el registry de otro integrante.

• CI: `calidad` y `pipeline` en verde en #34 y #38.

4. Hallazgos

• `mlflow` completo no es necesario para servir: basta `mlflow-skinny` y `psutil`.

• Podman no está instalado en la máquina de la auditoría; el `Containerfile` se probó con Docker.

• ONNX con el HGB cambia la predicción en parte de las filas por usar float32 (detalle en la sección 5).

• #35, #36 y #37 se fusionaron en la rama de #34 cuando esta ya estaba en `main`, así que su contenido se reunió en #38.

5. Extra: ONNX Runtime frente a sklearn

Modelo: champion de P1 (`mibici-equipo11-p1-demanda` v1, HistGradientBoostingRegressor con `StandardScaler` y `OneHotEncoder`), CPU local, 200 repeticiones, solo inferencia del modelo.

Filas	Motor	p50 ms	p95 ms	Filas/s
1	sklearn	5.09	6.88	196
1	ONNX Runtime	0.11	0.16	8 850
100	sklearn	5.27	6.44	18 977
100	ONNX Runtime	0.39	0.46	257 693
500	sklearn	7.56	11.67	66 116
500	ONNX Runtime	1.10	1.45	454 663

Aspecto	Detalle
Latencia	Menos de 1 ms por fila contra unos 5 ms de sklearn
Portabilidad y seguridad	El `.onnx` no depende de la versión de sklearn ni de pickle

Aspecto	Detalle
Tamaño del modelo	`model.onnx` 450 KB contra `model.pkl` 725 KB
Diferencias numéricas	En 20 000 filas de prueba, diferencia media 0.088 viajes, p99 1.11 y máxima 16.9; 25.7 % de las filas cambian más de 0.01
Error contra los viajes reales	MAE sklearn 5.698 y ONNX 5.693
Costo de mantenimiento	Reconvertir y revisar la paridad con cada reentrenamiento; dos caminos de inferencia

Recomendación: mantener sklearn como motor por defecto y usar ONNX para modelos lineales o con alto volumen de peticiones, una vez definida la tolerancia de diferencia. El análisis completo está en `docs/analisis_onnx.md`.

6. Aportes por integrante en esta entrega

Integrante	Aporte
Chuy	3 commits en `main` (PR #31 a #33: esquemas, carga del modelo y endpoints)
Jimena	Revisión y aprobación de #34
Luis	Sin commits en esta entrega
Oscar	1 commit en `main` (#34) y 5 en #38 (CI, ONNX, evidencia y reporte)

Evidencia

Pruebas

```
    warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
335 passed, 19 warnings in 100.91s (0:01:40)
```

Contenedor en ejecución

```
$ docker run -d --name mibici-api -p 8000:8000 mibici-api
596ec7e2e943
$ docker ps
NAMES        IMAGE        STATUS                    PORTS
mibici-api   mibici-api   Up 10 seconds (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
$ docker logs mibici-api
INFO:     Waiting for application startup.
2026-11-07 00:34:18.558 | INFO     | mi_bici.api.model:load_local_model:83 - Modelo local cargado: mibici-
equipo11-p1-demanda v1
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
imagen mibici-api 815MB
```

Peticiones a la API

```
$ curl http://127.0.0.1:8000/health
{"status":"ok"}
$ curl http://127.0.0.1:8000/model-info
{"model_name":"mibici-equipo11-p1-demanda","model_version":"1","run_id":"8b2ba4b2a00f4ccc9ec1422132f55c19"}
$ curl -X POST http://127.0.0.1:8000/p1/predict -H 'Content-Type: application/json' -d '{"instances":[{"lag_1d":12.0,"la
g_7d":11.0,"lag_14d":3.0,"lag_21d":4.0,"rolling_mean_7d":11.857143,"capacity":19,"day_of_week":2,"month":9,"is_weekend":
0,"is_holiday":0,"zone":"ZPN"}]}'
{"model_name":"mibici-equipo11-p1-demanda","model_version":"1","predictions":[10.182341634926482]}
HTTP 200
$ curl -X POST http://127.0.0.1:8000/p1/predict -d '{"instances":[{"month":13}]}'
{"detail":[{"loc":["body","instances",0,"lag_1d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"lag_7d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"lag_14d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"lag_21d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"rolling_mean_7d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"capacity"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"day_of_week"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"month"],"msg":"Input should be less than or equal to
12","type":"less_than_equal"},{"loc":["body","instances",0,"is_weekend"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"is_holiday"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"zone"],"msg":"Field required","type":"missing"}]}
HTTP 422
```

ONNX Runtime: paridad y latencia

```
$ python scripts/export_onnx.py
diferencia media: 0.0880  p99: 1.1093  máxima: 16.8949
$ python scripts/benchmark_inference.py
filas | motor   | p50 ms | p95 ms | filas/s
    1 | sklearn |   5.09 |   6.88 | 196
    1 | onnx    |   0.11 |   0.16 | 8850
  100 | sklearn |   5.27 |   6.44 | 18977
  100 | onnx    |   0.39 |   0.46 | 257693
  500 | sklearn |   7.56 |  11.67 | 66116
  500 | onnx    |   1.10 |   1.45 | 454663
```

API respondiendo desde el contenedor (Docker)

